

AICHE Website Project

The Complete Beginner's Backend Guide

Python + FastAPI + SQLAlchemy + PostgreSQL — every file, every line explained, every user role's journey, every endpoint, and the 2-/3-member split

This is the deepest version of the backend guide. Every code block has a comment on (almost) every line explaining what that line does and why it's there — written for someone who has never built a backend before. Nothing is assumed. If a concept (ORM, migration, JWT, dependency injection, environment variable...) shows up, it's explained in plain language the first time it appears, not just used. It also includes a dedicated walk-through of what happens on the backend for each of the 5 user types in the baseline.

Stack: **Python, FastAPI, SQLAlchemy (ORM), PostgreSQL (database), Alembic (migrations), Pydantic (validation), JWT + bcrypt (auth)** — 100% free and open source.

Contents

- 1. The Big Picture — how a request travels through your backend, in plain words
- 2. Backend Overview by User Role — what happens for Guest / Student / Member / Committee Admin / President-VP-Secretary
- 3. Stack, Concepts, & Environment Setup — explained one by one
- 4. Project Structure — what every folder is for
- 5. database.py — every line explained
- 6. models/ — every table, every column, every line explained
- 7. schemas/ — every line explained
- 8. dependencies.py — auth explained line by line
- 9. Core Backend Logic — the 5 trickiest rules, explained and coded
- 10. Routers — full code, every line explained
- 11. main.py — every line explained
- 12. Alembic Migrations & Seed Data — explained in depth
- 13. Full API Reference — every endpoint, every field explained
- 14. Running & Deploying for free
- 15. Backend Split — 2 Members
- 16. Backend Split — 3 Members
- 17. Git Workflow for This Stack
- 18. Common Beginner Mistakes & How to Avoid Them
- 19. Final Integration Checklist

1. The Big Picture — How a Request Travels Through Your Backend

Before touching any file, it helps to see the whole journey one request takes, from the moment someone calls your API to the moment they get an answer back. Every file in this guide is just one stop along this journey.

1. Someone (Postman, /docs, or later the frontend) sends an HTTP request, e.g.:
POST /api/courses/5/enroll
Headers: Authorization: Bearer eyJhbGciOi...
2. FastAPI (in main.py) looks at the URL and method, and routes it to the matching function inside routers/courses.py – this matching is called "routing".
3. Before your function's own code runs, FastAPI runs any Depends(...) you attached:
 - Depends(get_db) -> opens a database connection for this one request
 - Depends(get_current_user) -> reads the Authorization header, decodes the JWT, looks up the user row, and hands it to you
 - Depends(require_role(...)) -> on top of the above, rejects the request early (403 Forbidden) if the role isn't allowed
4. If the request has a body (like signup data), FastAPI reads the raw JSON and converts it into a Pydantic schema object (from schemas/) – if the JSON doesn't match the shape you defined, FastAPI rejects it automatically with a 422 error, before your code even runs. This is "validation".
5. Your function's own code runs. It typically:
 - a) uses the SQLAlchemy `db` session to query/insert/update rows (using classes from models/, which represent real tables in PostgreSQL)
 - b) does any business logic (e.g. "is the previous course part completed?")
 - c) returns a Python object (e.g. a SQLAlchemy model instance, or a dict)
6. FastAPI converts whatever you returned into JSON, using the `response\_model` schema you declared on the route – this also means fields NOT listed in that schema (like password_hash) never leak out, even if you forgot to remove them from what you returned.
7. The JSON response is sent back over HTTP to whoever made the request.

Every section below is really just zooming into one of these 7 steps and showing you exactly how it's written in code.

2. Backend Overview by User Role

This section is a map of the whole backend from a completely different angle: instead of "here's what this file does," it's "here's what happens on the server for this specific kind of person, from the moment they open the app to whatever they're trying to do." Read this section first if you want the big picture before diving into code, or last as a way to double-check the code sections above actually cover everything each role needs.

2.1 Guest — not logged in at all

A Guest never sends an Authorization header, so every route they touch must be one with no Depends(get_current_user) at all — if a Guest tries to call a protected route, get_current_user() fails immediately with 401 Unauthorized because there's no token to decode.

What the Guest does	Endpoint(s)	What happens on the backend
Browse courses/workshops/events	GET /api/courses, /api/workshops, /api/events	Public routes — no auth dependency at all, straight database read, returned through the *Out schemas.
Try to enroll / register	POST /api/courses/{id}/enroll (etc.)	Blocked: get_current_user() has no token to check, so FastAPI returns 401 Unauthorized before any route logic runs.
Sign up	POST /api/auth/signup	The ONE write action a Guest can trigger — creates a new users row with role defaulting to student.

2.2 Student — has an account, no committee

A Student's JWT contains role: "student" and committee_id: null. Every route they use either has no role restriction (just Depends(get_current_user)) or explicitly allows "student" in its require_role(...) list.

What the Student does	Endpoint(s)	What happens on the backend
Log in	POST /api/auth/login	Password checked via bcrypt, JWT issued containing their user_id and role.
Enroll in a course, complete parts in order	POST /api/courses/{id}/enroll, POST /api/courses/course-parts/{id}/complete	get_current_user() confirms identity; is_part_unlocked() (Section 9.1) enforces the sequential rule specifically for THIS user_id.
Earn a certificate	GET /api/courses/{id}/certificate	check_and_issue_course_certificate() checks every course_parts row for this user before returning a PDF link — see Section 9.3.
Register for events, attend workshops	POST /api/events/{id}/register, workshop attendance routes	Same identity + role checks; attendance itself is only ever written by an Admin, never by the Student themselves.
View their own profile	GET /api/auth/me	Returns exactly the row matching their own token's user_id — a Student can never fetch another user's private data through this route, since it always uses current_user, never a client-supplied id.
Try to view Sessions	GET /api/sessions	Blocked by require_role in that router — Section 2.3 baseline rule: Students never see committee Sessions at all, only Members and above.

2.3 Member — belongs to a committee

A Member's JWT additionally carries a real `committee_id`. This is the role where the backend does the most extra filtering work, because almost every Member-facing route needs to answer "yes, but only THEIR committee's data."

What the Member does	Endpoint(s)	What happens on the backend
Everything a Student can do	(see above)	Same routes, same logic — a Member is a superset of Student, enforced simply by including "member" in the same <code>require_role(...)</code> lists as "student" wherever they overlap.
View Sessions	GET <code>/api/sessions</code>	<code>get_visible_sessions()</code> (Section 9.4) filters by <code>current_user.committee_id</code> + public sessions — the Member never even receives other committees' <code>committee_specific</code> sessions in the response, they're excluded at the database query level, not just hidden in a UI.
See their Total Points / Point History	GET <code>/api/points/{id}/total</code> , <code>/history</code>	<code>get_total_points()</code> runs a live SUM over <code>point_entries</code> filtered to their own <code>user_id</code> .
View the Committee Database	GET <code>/api/committees/{id}/database</code>	Allowed for ANY <code>committee_id</code> (view-all rule) — this is one of the few places a Member can read data belonging to a committee that isn't their own.
Edit their OWN committee's Database	POST <code>/api/committees/{id}/database</code>	<code>has_edit_access()</code> (Section 8.3) checks <code>committee_id</code> in the URL against <code>current_user.committee_id</code> (or an <code>AccessPermission</code> override) before allowing the write.
Leave Feedback	POST <code>/api/feedback</code>	Same as Student — any logged-in user can leave feedback; the <code>target_type/target_id</code> just record what it's about.

2.4 Committee Admin — the shared committee login

A Committee Admin logs in with a shared committee email (Section 5 of the baseline), and their user row's `committee_id` identifies which committee they're managing content for. The backend treats them almost identically to a Member for viewing, but unlocks the create/edit endpoints across every module.

What the Committee Admin does	Endpoint(s)	What happens on the backend
Create/edit Courses, Workshops, Events, Sessions for their committee	POST <code>/api/courses</code> , <code>/api/workshops</code> , <code>/api/events</code> , <code>/api/sessions</code> (and their sub-resources)	<code>require_role("committee_admin", ...)</code> passes the role check; <code>created_by</code> is stamped with their own <code>user_id</code> automatically.
Record Attendance	POST <code>.../attendance</code> routes across Courses, Workshops, Events, Sessions	Same role check; these routes also call <code>add_point_entry()</code> where the baseline's scoring rules apply (Sections 35-39).
Try to edit ANOTHER committee's section without permission	e.g. POST <code>/api/committees/{other_id}/database</code>	<code>has_edit_access()</code> returns False (no matching <code>AccessPermission</code> row with <code>can_edit=True</code>) → 403 Forbidden, even though the role check alone would have passed.

What the Committee Admin does	Endpoint(s)	What happens on the backend
Edit a section after being granted Edit Access	same as above, after President/VP/Secretary calls POST /api/access-permissions for them	has_edit_access() now finds a matching row with can_edit=True and allows it — no code change needed, just new data.

2.5 QC Admin — the one deliberate exception

QC is technically a Committee Admin, but the baseline (Section 33) gives it one special power: visibility across every committee's Sessions, plus the ability to manage scoring everywhere, not just for QC's own committee.

What QC does differently	Endpoint(s)	What happens on the backend
Sees every committee's Sessions	GET /api/sessions	get_visible_sessions() has an explicit early check: if <code>current_user.committee.name == "QC"</code> : return everything, before the normal <code>committee_id</code> filter ever runs (Section 9.4).
Records scoring for any committee's session	POST /api/sessions/{id}/scoring	The role check allows "committee_admin"; because QC can already SEE every session (above), nothing further blocks them from scoring it.
Uploads Monthly Reports for Members	POST /api/reports/members/{id}/monthly-report	require_role restricted specifically to QC (checked via committee name, same pattern as the Sessions check).

2.6 President / Vice President / Secretary — full access

These three roles share one login email (Section 6 of the baseline) and are, by design, allowed almost everywhere in `require_role(...)` lists across the whole codebase. Their extra, EXCLUSIVE powers are the ones worth tracking closely, since they touch other people's data directly.

Exclusive power	Endpoint(s)	What happens on the backend
Grant/revoke a committee's Edit Access	POST /api/access-permissions	Writes/updates an AccessPermission row — this single row is what <code>has_edit_access()</code> checks everywhere else in the system (Section 8.3). No other role can call this endpoint at all.
Approve deletion of a reported Feedback comment	DELETE /api/feedback/{id}	<code>delete_reported_feedback()</code> (Section 9.5) — deletes the comment AND the author's account together, in a single committed transaction, restricted to these 3 roles.
View/edit the General Committee database	GET/POST /api/committees/{general_id}/database	Same database routes as any committee, but this <code>committee_id</code> is never returned to Members/Students in any dashboard-summary endpoint — it's only reachable if you already know its id and have the right role.
Full oversight (Users, all Committees, all Databases)	Various GET routes across every router	Simply included in every <code>require_role(...)</code> list — there is deliberately no separate "admin panel" codebase, just the same routers with a role that happens to be allowed everywhere.

Notice the pattern across all 5 roles: the backend never has role-specific ROUTES — it has the same routes for everyone, and each one decides who's allowed in using the exact same two tools every time: `Depends(require_role(...))` for role checks, and `has_edit_access()` for the one extra committee-ownership rule. Once you understand those two mechanisms (Sections 8.2 and 8.3), you already understand how every role's access is enforced everywhere in the project.

3. Stack, Concepts & Environment Setup — Explained One by One

3.1 What each tool actually does (in plain words)

Tool	What it is, explained simply
Python	The programming language everything below is written in.
FastAPI	A library that turns your Python functions into a web server that understands HTTP requests — it also auto-generates a testable docs page at /docs.
SQLAlchemy (ORM)	"ORM" = Object-Relational Mapper. Instead of writing raw SQL like INSERT INTO users (...) VALUES (...), you write db.add(User(...)) — SQLAlchemy translates your Python objects into SQL behind the scenes.
PostgreSQL	The actual database program that stores your data permanently on disk (or in the cloud). SQLAlchemy talks to it; you almost never touch it directly.
Alembic	Tracks changes to your database structure over time as small, ordered files ("migrations"), so your whole team's databases stay in sync.
Pydantic	Defines exactly what shape of JSON you expect in and out of your API, and automatically checks/rejects anything that doesn't match.
JWT (JSON Web Token)	A signed piece of text your server hands out after login. The client sends it back on every future request to prove who they are, without the server needing to remember every logged-in session in memory.
bcrypt	A one-way hashing algorithm for passwords — turns "mypassword123" into an unreadable string that can be checked but never reversed back to the original.
Uvicorn	The actual program that runs your FastAPI app and listens for incoming network connections — FastAPI itself is just Python code until Uvicorn serves it.
.env file	A plain text file holding secret values (database password, JWT secret) that should never be typed directly into your code or uploaded to GitHub.

3.2 Why this combination, specifically, for beginners

- FastAPI gives you the /docs page for free — you can test every endpoint by clicking buttons in the browser before you've written a single line of frontend code.
- SQLAlchemy + Alembic means you never hand-write CREATE TABLE SQL — you describe tables as Python classes and let the tools generate and apply the SQL for you.
- Every single package listed is free forever and open source — nothing here requires a credit card, even in production, as long as you stay on free hosting tiers.

3.3 Installing everything — line by line

```
python -m venv venv
# Creates an isolated folder called "venv" containing its own copy of Python and pip.
# Isolated = installing packages here doesn't affect other Python projects on your computer.

source venv/bin/activate
# Windows: venv\Scripts\activate
# "Activates" that isolated environment for your current terminal session – from now on,
# `pip install` and `python` both refer to the copies inside venv/, not your system Python.

pip install fastapi uvicorn[standard] sqlalchemy psycopy2-binary \
    python-dotenv pydantic[email] passlib[bcrypt] python-jose[cryptography] alembic
# pip = Python's package installer. Each name after "install" is a free open-source package:
# fastapi the web framework itself
# uvicorn[standard] the server that runs FastAPI ([standard] adds extra performance extras)
# sqlalchemy the ORM
# psycopy2-binary the low-level driver that lets Python actually talk to PostgreSQL
# python-dotenv reads your .env file into Python
# pydantic[email] validation library ([email] adds EmailStr type checking)
# passlib[bcrypt] password hashing
# python-jose[cryptography] creates and verifies JWT tokens
# alembic database migration tool
```

3.4 requirements.txt — so your teammates install the exact same versions

```
pip freeze > requirements.txt
# "pip freeze" prints every installed package AND its exact version number.
# ">" writes that output into a new file called requirements.txt instead of the screen.
# Commit this file to Git. A teammate then only needs to run:
# pip install -r requirements.txt
# and pip installs the exact same versions you used – this avoids the classic
# "it works on my machine" bug caused by mismatched package versions.
```

3.5 .env — explained field by field

```
DATABASE_URL=postgresql://username:password@host:5432/aiche_db
# The full address SQLAlchemy uses to connect to PostgreSQL:
# postgresql:// <- tells SQLAlchemy which database engine to use
# username:password <- your database login credentials (from Neon/Supabase/local Postgres)
# @host:5432 <- where the database server lives, and its port (5432 is Postgres's default)
# /aiche_db <- the specific database name inside that server

JWT_SECRET=replace-with-a-long-random-string
# A secret key used to "sign" every JWT token, so the server can later verify a token wasn't
# tampered with. If someone else learns this string, they could forge fake login tokens –
# treat it like a master password. Generate one with: python -c "import secrets; print(secrets.token_hex(32))"

JWT_ALGORITHM=HS256
# The specific mathematical algorithm used to sign/verify tokens. HS256 is the standard,
# simplest choice and is what python-jose defaults to.

ACCESS_TOKEN_EXPIRE_MINUTES=1440
# How long a login token stays valid before the user must log in again. 1440 minutes = 24 hours.
```

Important: add a line containing just `.env` to a file called `.gitignore` in your project root. This tells Git to never upload your secrets to GitHub, even accidentally.

4. Project Structure — What Every Folder Is For

```
backend/
■■■ .env <- your secret values (never committed to Git)
■■■ .gitignore <- tells Git which files/folders to ignore (.env, venv/, __pycache__/)
■■■ requirements.txt <- exact list of packages + versions, for teammates to install
■■■ alembic.ini <- Alembic's configuration file (created by `alembic init`)
■■■ alembic/
■ ■■■ env.py <- tells Alembic where your models and database are
■ ■■■ versions/ <- one auto-generated file per database change, in order
■■■ app/
■ ■■■ __init__.py <- empty file that tells Python "this folder is a package"
■ ■■■ database.py <- opens the connection to PostgreSQL (Section 5)
■ ■■■ models/ <- one file per group of tables (Section 6)
■ ■■■ schemas/ <- one file per group of request/response shapes (Section 7)
■ ■■■ dependencies.py <- auth + role-checking logic reused by every route (Section 8)
■ ■■■ core_logic.py <- the business-rule functions from Section 9
■ ■■■ routers/ <- one file per feature, each with its own URL prefix (Section 10)
■ ■■■ main.py <- creates the app and plugs every router into it (Section 11)
■■■ seed.py <- a one-time script that creates starter data (Section 12)
```

Why split into so many small files instead of one giant file? Two reasons: (1) it's much easier to find and understand 40 lines about "courses" than to scroll through 2,000 lines of everything mixed together, and (2) when working in a team, two people editing two different files essentially never cause a Git merge conflict — see Sections 15–17.

5. database.py — Every Line Explained

```
import os
# Python's built-in module for reading environment variables (values from the .env file).

from dotenv import load_dotenv
# The function that actually reads your .env file and loads its values into the environment.

from sqlalchemy import create_engine
# create_engine builds the low-level connection object SQLAlchemy uses to talk to Postgres.

from sqlalchemy.orm import sessionmaker, declarative_base
# sessionmaker -> a factory that creates new "Session" objects (a session = one conversation
# with the database: some queries, maybe some writes, then closed)
# declarative_base -> a function that returns a base class; every one of your table classes
# in models/ will inherit from this base, which is how SQLAlchemy knows
# "this Python class represents a database table"

load_dotenv()
# Actually reads the .env file in your project folder and makes its values available via os.getenv()

DATABASE_URL = os.getenv("DATABASE_URL")
# Reads the DATABASE_URL value from .env into a normal Python variable.

engine = create_engine(DATABASE_URL)
# Creates the actual connection engine, pointed at your Postgres database.
# This does NOT connect immediately – it connects lazily, the first time it's actually used.

SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)
# Builds a "session factory" bound to our engine.
# autocommit=False -> nothing is saved to the database until you explicitly call db.commit()
# autoflush=False -> SQLAlchemy won't silently send half-finished changes to the database
# before you're ready; you stay in control of exactly when writes happen

Base = declarative_base()
# Creates the shared base class. Every model in models/ will do "class User(Base): ..." –
# this is what lets SQLAlchemy (and later Alembic) discover every table in your project.

def get_db():
    # This function is what FastAPI calls automatically via Depends(get_db) on every route
    # that needs database access. It's written as a "generator" (notice the `yield` below)
    # so FastAPI can guarantee cleanup happens even if your route code raises an error.
```

```
db = SessionLocal()
# Opens one new database session for this one incoming request.
try:
    yield db
    # Hands that session to your route function. Your route's code runs here, using `db`.
finally:
    db.close()
    # No matter what happened above (success or an error), always close the session
    # afterwards, so we don't leak open database connections over time.
```

6. models/ — Every Table, Every Column, Every Line Explained

Each file below defines one or more SQLAlchemy "model" classes. Every model class = one real table that will exist inside your PostgreSQL database once you run `alembic upgrade head` (Section 12).

6.1 models/user.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, DateTime, Enum
# Column -> declares one column in a table
# Integer, String, DateTime -> the data TYPE of that column (whole number, text, date+time)
# ForeignKey -> marks a column as "points to the id column of another table" (a relationship)
# Enum -> a column that can only ever hold one of a fixed list of values

from sqlalchemy.orm import relationship
# relationship() doesn't create a real database column – it creates a convenient Python
# shortcut, e.g. so you can write `user.committee.name` instead of writing a manual JOIN query.

from app.database import Base
# Imports the shared Base class from database.py – every model below inherits from it.

import enum, datetime
# enum -> Python's built-in tool for defining a fixed set of named choices
# datetime -> used below to generate the current date/time as a default value

class RoleEnum(str, enum.Enum):
    # Defines the fixed list of roles allowed anywhere the User.role column is used.
    # Inheriting from `str` as well as `enum.Enum` means these values behave like normal
    # strings when converted to/from JSON, which keeps the API responses simple.
    student = "student"
    member = "member"
    committee_admin = "committee_admin"
    president = "president"
    vice_president = "vice_president"
    secretary = "secretary"

class Committee(Base):
    # Represents the "committees" table (Marketing, OC, Technical, etc. from the baseline).
    __tablename__ = "committees"
    # The literal name this table will have inside PostgreSQL.

    id = Column(Integer, primary_key=True)
    # The unique identifier for each row. primary_key=True means Postgres auto-generates a
    # new number (1, 2, 3...) every time a new committee is inserted, and no two rows can
    # ever share the same id.

    name = Column(String, unique=True)
    # The committee's name, e.g. "Technical". unique=True means the database itself will
    # reject any attempt to insert a second committee with the same name.

    login_email = Column(String, unique=True)
    # The shared login email for that committee, e.g. AICHECU.Marketing@gmail.com.

    users = relationship("User", back_populates="committee")
    # A Python-only shortcut: committee.users gives you a list of every User row whose
    # committee_id points at this committee. "back_populates" links this to the matching
    # relationship() defined on the User class below, so both directions stay in sync.

class User(Base):
    # Represents the "users" table – every Student, Member, Committee Admin, President, etc.
    __tablename__ = "users"

    id = Column(Integer, primary_key=True, index=True)
    # index=True tells Postgres to build a fast lookup index on this column – since we look
    # up users by id constantly (on every authenticated request), this keeps things fast.

    full_name = Column(String)
    username = Column(String, unique=True, index=True)
    email = Column(String, unique=True, index=True)
    # unique=True + index=True on username/email: no two users can share one, and looking
    # someone up by email (e.g. during login) stays fast even with thousands of rows.
```

```
password_hash = Column(String)
# NEVER a column called "password" – we only ever store the bcrypt-hashed version,
# created in dependencies.py, never the real password the user typed.

role = Column(Enum(RoleEnum), default=RoleEnum.student)
# Restricts this column to only the values defined in RoleEnum above. If new code
# somehow tries to save role="wizard", the database rejects it. default=RoleEnum.student
# means if nothing is specified, new accounts start as a Student.

committee_id = Column(Integer, ForeignKey("committees.id"), nullable=True)
# ForeignKey("committees.id") means this column must either be empty or match an
# existing committees.id value – the database enforces this automatically.
# nullable=True means it's allowed to be empty (Students/Guests have no committee).

created_at = Column(DateTime, default=datetime.datetime.utcnow)
# default=datetime.datetime.utcnow means: if you don't provide a value when creating a
# user, SQLAlchemy automatically fills in "right now" (in UTC time) for you.

committee = relationship("Committee", back_populates="users")
# The other half of the shortcut above: user.committee gives you the full Committee
# object (not just the id), e.g. user.committee.name.
```

6.2 models/course.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, DateTime, Float, Enum
from sqlalchemy.orm import relationship
from app.database import Base
import enum, datetime

class PartType(str, enum.Enum):
    # The 4 allowed kinds of "part" inside a Course, matching the baseline's fixed path:
    # Lecture -> Task -> Quiz -> ... -> Final Exam.
    lecture = "lecture"
    task = "task"
    quiz = "quiz"
    final_exam = "final_exam"

class ProgressStatus(str, enum.Enum):
    # The 3 states a student's progress on one course part can be in.
    locked = "locked" # can't be opened yet – previous part isn't done
    in_progress = "in_progress" # opened but not finished (used for quizzes/tasks in draft)
    completed = "completed" # done – this is what unlocks the NEXT part

class Course(Base):
    __tablename__ = "courses"
    id = Column(Integer, primary_key=True)
    title = Column(String)
    description = Column(String)
    image_url = Column(String) # link to the image, uploaded separately to Cloudinary
    instructor = Column(String)
    level = Column(String) # free text like "Beginner" – kept simple on purpose
    duration = Column(String) # free text like "4 weeks"
    created_by = Column(Integer, ForeignKey("users.id"))
    # Records WHICH admin created this course – useful for auditing/history later.

    parts = relationship("CoursePart", back_populates="course",
                        order_by="CoursePart.order_index")
    # course.parts gives you every CoursePart belonging to this course, automatically
    # sorted by order_index – so you never have to remember to sort them yourself.

class CoursePart(Base):
    __tablename__ = "course_parts"
    id = Column(Integer, primary_key=True)
    course_id = Column(Integer, ForeignKey("courses.id"))
    # Which course this part belongs to.

    type = Column(Enum(PartType))
    # lecture / task / quiz / final_exam – restricted to the PartType list above.

    order_index = Column(Integer)
    # The position of this part in the sequence: 1, 2, 3... This single number is what
    # powers the entire "sequential unlock" rule explained in Section 9.1.

    title = Column(String)
    content_url = Column(String, nullable=True)
    # Where the actual content lives (a video link, a PDF, or JSON describing quiz questions).
    # nullable=True because a Final Exam might not need a content_url at all.

    course = relationship("Course", back_populates="parts")

class CourseProgress(Base):
    # One row per (student, course_part) pair – this is how we know, for every single
    # student, exactly which parts of which courses they've finished.
    __tablename__ = "course_progress"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    course_part_id = Column(Integer, ForeignKey("course_parts.id"))
    status = Column(Enum(ProgressStatus), default=ProgressStatus.locked)
    completion_type = Column(String, nullable=True)
    # "online" if the student completed it themselves in the system, or
    # "offline_confirmed" if an admin marked them Present on a physical attendance sheet.
    # Both values mean the same thing to the unlock-checking code (Section 9.1) – it only
    # cares about `status`, not how it got there.
    completed_at = Column(DateTime, nullable=True)
```

```
# When it was completed. Left empty (nullable) until status becomes "completed".
```

```
class Certificate(Base):
    __tablename__ = "certificates"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    type = Column(String) # "course" or "workshop" – tells you how to interpret source_id
    source_id = Column(Integer)
    # The id of the specific course or workshop this certificate is for. We don't use a
    # ForeignKey here on purpose, since it could point to either the courses table or the
    # workshops table depending on `type` – a plain Integer keeps that flexible.
    grade = Column(Float, nullable=True)
    # Only filled in for course certificates (the Final Exam grade). Workshop certificates
    # leave this empty since Section 21 of the baseline says workshop certificates are
    # attendance-based, not grade-based.
    pdf_url = Column(String)
    # Where the generated PDF file is stored (Cloudinary link).
    issued_at = Column(DateTime, default=datetime.datetime.utcnow)
```

6.3 models/workshop.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, DateTime
from app.database import Base

class Workshop(Base):
    __tablename__ = "workshops"
    id = Column(Integer, primary_key=True)
    title = Column(String)
    description = Column(String)
    image_url = Column(String)
    instructor = Column(String) # the "Instructor / Speaker" field from the baseline
    created_by = Column(Integer, ForeignKey("users.id"))
    # Note: no `parts` relationship here like Course has – Workshops don't follow a fixed
    # sequential path (Section 19 of the baseline says "no Path Learning"), they just have
    # one or more independent Sessions, defined below.

class WorkshopSession(Base):
    # A single meeting/session that belongs to a Workshop. A workshop can have 1 or many.
    __tablename__ = "workshop_sessions"
    id = Column(Integer, primary_key=True)
    workshop_id = Column(Integer, ForeignKey("workshops.id"))
    title = Column(String)
    description = Column(String)
    date = Column(DateTime)
    mode = Column(String) # "online" or "offline"
    location_or_link = Column(String) # a physical address OR a Zoom/Meet link, depending on mode
    materials_url = Column(String, nullable=True)
    # Optional link to slides/materials for this specific session.

class WorkshopAttendance(Base):
    # One row per (student, workshop_session) – tracks who showed up to which session.
    # This is what qualifies someone for the Attendance Certificate (Section 21).
    __tablename__ = "workshop_attendance"
    id = Column(Integer, primary_key=True)
    workshop_session_id = Column(Integer, ForeignKey("workshop_sessions.id"))
    user_id = Column(Integer, ForeignKey("users.id"))
    status = Column(String) # "present" or "absent"
    recorded_by = Column(Integer, ForeignKey("users.id"))
    # WHICH admin recorded this attendance – useful if a dispute comes up later.
```

6.4 models/event.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, DateTime
from app.database import Base

class EventSiteVisit(Base):
    # One table handles BOTH Events and Site Visits (Section 22 of the baseline explicitly
    # merges them), distinguished only by the `type` column – this avoids duplicating
    # almost-identical columns across two separate tables.
    __tablename__ = "events_site_visits"
    id = Column(Integer, primary_key=True)
    type = Column(String) # "event" or "site_visit"
    title = Column(String)
    description = Column(String)
    date = Column(DateTime)
    location = Column(String)
    registration_deadline = Column(DateTime)
    # Used to automatically calculate the Upcoming / Closed Registration / Finished status
    # described in Section 24 – we compare this and `date` against the current time whenever
    # we return this row, instead of storing a "status" column that could go stale.
    created_by = Column(Integer, ForeignKey("users.id"))

class EventRegistration(Base):
    # One row per (student, event) – who signed up for what.
    __tablename__ = "event_registrations"
    id = Column(Integer, primary_key=True)
    event_id = Column(Integer, ForeignKey("events_site_visits.id"))
    user_id = Column(Integer, ForeignKey("users.id"))
    registered_at = Column(DateTime)

class EventAttendance(Base):
```

```
# One row per (student, event) – whether they actually showed up, filled in by an
# admin after the event happens. This feeds directly into the Point System (Section 37).
__tablename__ = "event_attendance"
id = Column(Integer, primary_key=True)
event_id = Column(Integer, ForeignKey("events_site_visits.id"))
user_id = Column(Integer, ForeignKey("users.id"))
status = Column(String) # "attended" | "absent_excused" | "absent_unexcused"
```

6.5 models/session.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, DateTime, Table
from sqlalchemy.orm import relationship
from app.database import Base

session_visibility_committees = Table(
    "session_visibility_committees", Base.metadata,
    Column("session_id", Integer, ForeignKey("committee_sessions.id")),
    Column("committee_id", Integer, ForeignKey("committees.id")),
)
# This is an "association table" – it's not a full model class with its own id, just a
# plain two-column table that links Sessions to Committees. We need it because a single
# committee_specific Session might target MORE than one committee at once (many-to-many),
# which a normal single committee_id foreign key on CommitteeSession couldn't express.

class CommitteeSession(Base):
    __tablename__ = "committee_sessions"
    id = Column(Integer, primary_key=True)
    committee_id = Column(Integer, ForeignKey("committees.id"))
    # The committee that CREATED this session (always exactly one).
    title = Column(String)
    description = Column(String)
    date = Column(DateTime)
    visibility = Column(String) # "public" or "committee_specific" – see Section 30 of baseline
    status = Column(String) # a simple free-text status like "scheduled" or "completed"
    created_by = Column(Integer, ForeignKey("users.id"))

    target_committees = relationship("Committee", secondary=session_visibility_committees)
    # `secondary=` tells SQLAlchemy to use the association table above to figure out the
    # many-to-many relationship. session.target_committees then gives you a list of every
    # Committee this session was specifically shared with (only relevant if visibility is
    # "committee_specific" and it was shared beyond just the creating committee).

class SessionScoring(Base):
    # One row per (member, session) – this is the literal table shown in Section 32 of the
    # baseline (Attendance / Tasks / Engagement / Bonus / Total columns).
    __tablename__ = "session_scoring"
    id = Column(Integer, primary_key=True)
    session_id = Column(Integer, ForeignKey("committee_sessions.id"))
    user_id = Column(Integer, ForeignKey("users.id"))
    attendance_status = Column(String)
    # One of: on_time | late_under_15 | late_over_15 | absent_excused | absent_unexcused
    task_status = Column(String)
    # One of: on_time | late | not_submitted
    engagement_status = Column(String)
    # One of: engaged | not_engaged
    bonus_points = Column(Integer, default=0)
    # A manually-added extra amount, entered by whoever is responsible for tracking points.
    recorded_by = Column(Integer, ForeignKey("users.id"))
    # Who filled in this row (for accountability).
```

6.6 models/point.py

```
from sqlalchemy import Column, Integer, String, ForeignKey, DateTime, Boolean, JSON
from app.database import Base
import datetime

class PointEntry(Base):
    # THE most important table in the whole system. Every single point a Member ever
    # earns or loses is one row here – never a number that gets directly edited.
    __tablename__ = "point_entries"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    source_type = Column(String)
    # "task" | "attendance" | "event" | "engagement" | "bonus" – which of the 5 scoring
    # rules from Section 35-39 of the baseline produced this entry.
    source_id = Column(Integer)
    # The id of the specific SessionScoring row or EventAttendance row this came from –
    # lets you trace any point entry back to exactly what earned/lost it.
    points = Column(Integer)
    # Can be POSITIVE (e.g. +2 for "Submitted on time") or NEGATIVE (e.g. -1 for "Not
```

```

# submitted") – see the exact numbers in the baseline's scoring tables (Sections 35-39).
reason = Column(String)
# A short human-readable label, e.g. "Task submitted on time" – this is exactly what
# shows up in the Member's Point History (Section 41 of the baseline).
created_at = Column(DateTime, default=datetime.datetime.utcnow)

```

```

class CommitteeDatabaseRow(Base):
    # Implements the spreadsheet-like Database feature (Sections 44-47). Instead of a
    # rigid table with fixed columns (Place, Phone, Price...), we use one flexible JSON
    # column – this means every committee can have totally different "columns" in their
    # database without needing a separate SQL table per committee.
    __tablename__ = "committee_databases"
    id = Column(Integer, primary_key=True)
    committee_id = Column(Integer, ForeignKey("committees.id"))
    row_data = Column(JSON)
    # e.g. {"place": "Cairo Print Shop", "phone": "0100...", "price": "500 EGP"} – any
    # shape the committee needs, stored as-is and returned as-is.
    created_by = Column(Integer, ForeignKey("users.id"))
    updated_at = Column(DateTime, default=datetime.datetime.utcnow)

```

```

class MonthlyReport(Base):
    __tablename__ = "monthly_reports"
    id = Column(Integer, primary_key=True)
    member_id = Column(Integer, ForeignKey("users.id"))
    # WHO the report is about (a specific Member).
    file_url = Column(String)
    # Where the uploaded report file lives (Cloudinary).
    month = Column(String)
    # Stored as plain text like "2026-09" for simplicity – easy to sort and filter.
    uploaded_by = Column(Integer, ForeignKey("users.id"))
    # The QC admin who uploaded it (per Section 42, QC uploads it, not the member).
    uploaded_at = Column(DateTime, default=datetime.datetime.utcnow)

```

```

class Feedback(Base):
    __tablename__ = "feedback"
    id = Column(Integer, primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id"))
    # The AUTHOR of the feedback comment.
    target_type = Column(String) # "course" | "workshop" | "event" | "site_visit"
    target_id = Column(Integer)
    # Same pattern as Certificate.source_id – a plain Integer that means different things
    # depending on target_type, kept flexible on purpose.
    comment = Column(String)
    is_reported = Column(Boolean, default=False)
    # Flipped to True when OC reports it (Section 49) – the comment stays visible, this
    # is just a flag that puts it in the President/VP's review queue.
    reported_by = Column(Integer, ForeignKey("users.id"), nullable=True)
    is_deleted = Column(Boolean, default=False)
    # A "soft delete" – we don't actually remove the row from the database, we just hide
    # it from the UI/API. This preserves a record of what was said and who approved removal.
    deleted_by = Column(Integer, ForeignKey("users.id"), nullable=True)
    created_at = Column(DateTime, default=datetime.datetime.utcnow)

```

```

class AccessPermission(Base):
    # Implements the Committee Access System (Section 7) – lets President/VP/Secretary
    # grant a committee edit rights on a section it doesn't normally own.
    __tablename__ = "access_permissions"
    id = Column(Integer, primary_key=True)
    section_name = Column(String)
    # A plain string identifying the section, e.g. "courses" or "events" – every router
    # that needs to respect this checks for a matching row (see Section 8.3 for the pattern).
    committee_id = Column(Integer, ForeignKey("committees.id"))
    can_edit = Column(Boolean, default=False)
    granted_by = Column(Integer, ForeignKey("users.id"))
    updated_at = Column(DateTime, default=datetime.datetime.utcnow)

```

6.7 models/ __init__.py — why this file matters

Alembic (Section 12) figures out what tables need to be created by inspecting `Base.metadata` — but that only contains a model if that model's file has actually been *imported* somewhere. This file exists purely to import every model class in one place, so nothing gets silently forgotten.

```
from app.models.user import User, Committee
from app.models.course import Course, CoursePart, CourseProgress, Certificate
from app.models.workshop import Workshop, WorkshopSession, WorkshopAttendance
from app.models.event import EventSiteVisit, EventRegistration, EventAttendance
from app.models.session import CommitteeSession, SessionScoring
from app.models.point import (PointEntry, CommitteeDatabaseRow, MonthlyReport,
                              Feedback, AccessPermission)

# Nothing else needs to happen in this file — just importing each class here is enough to
# register it with Base.metadata, because the `class X(Base):` line runs as soon as the
# file is imported.
```

7. schemas/ — Every Line Explained

Quick reminder of the difference, since it trips up almost every beginner: **models/** describes what a table looks like *INSIDE* the database. **schemas/** describes what JSON is allowed to come *IN* and what JSON you promise to send *OUT* through the API — they often look similar but serve completely different jobs.

7.1 schemas/user.py

```
from pydantic import BaseModel, EmailStr
# BaseModel -> the parent class every schema inherits from; gives you automatic validation.
# EmailStr -> a special string type that validates the value actually looks like an email
# address (has an @, a domain, etc.) — rejects bad input automatically.

from typing import Optional
# Optional[X] means "this field is either type X, or it can be missing/None".

from datetime import datetime
# Used below so Pydantic knows created_at should be treated as a real datetime, not text.

class UserCreate(BaseModel):
    # Describes exactly what JSON body /api/auth/signup expects to receive.
    full_name: str
    username: str
    email: EmailStr
    password: str
    # Notice: no "role" or "committee_id" here — a new signup can NEVER set their own role;
    # that keeps the "Member ability to self-assign a committee" bug the baseline explicitly
    # warns against (Section 57) impossible at the schema level, not just by convention.

class UserLogin(BaseModel):
    # Describes what JSON /api/auth/login expects.
    email: EmailStr
    password: str

class UserOut(BaseModel):
    # Describes what JSON the API sends BACK whenever it returns a user — notice there is
    # no password_hash field here at all. Even if a route accidentally tried to return the
    # full User model object including password_hash, FastAPI would strip it out, because
    # it only keeps fields that are declared in this UserOut schema.
    id: int
    full_name: str
    username: str
    email: str
    role: str
    committee_id: Optional[int] = None
    # = None means: if this field is missing (e.g. for a Guest-created model), default to
    # None instead of raising an error.
    created_at: datetime

class Config:
    from_attributes = True
```

```

# This tells Pydantic "it's OK to build this schema directly from a SQLAlchemy
# model object" (e.g. UserOut.from_orm(some_user_row)) instead of requiring a plain
# dictionary – without this line, returning a SQLAlchemy object from a route with
# response_model=UserOut would fail.

```

```

class Token(BaseModel):
    # Describes what JSON /api/auth/login sends back on success.
    access_token: str
    token_type: str = "bearer"
    # A default value – every route that returns a Token will say token_type: "bearer"
    # unless you explicitly override it, which you never need to for this project.

```

7.2 schemas/course.py

```

from pydantic import BaseModel
from typing import Optional
from datetime import datetime

class CourseCreate(BaseModel):
    title: str
    description: str
    image_url: Optional[str] = None
    # Optional because an admin might create a course and add the image afterwards.
    instructor: str
    level: str
    duration: str
    # Note again: no `created_by` here – that value comes from the LOGGED-IN admin
    # (current_user.id inside the route, see Section 10), never from the request body.
    # This prevents someone from pretending a course was created by a different admin.

class CourseOut(BaseModel):
    id: int
    title: str
    description: str
    instructor: str
    level: str
    duration: str
    created_by: int
    class Config:
        from_attributes = True

class CoursePartCreate(BaseModel):
    type: str # "lecture" | "task" | "quiz" | "final_exam"
    order_index: int
    title: str
    content_url: Optional[str] = None

class CoursePartOut(CoursePartCreate):
    # Inheriting from CoursePartCreate means CoursePartOut automatically gets all 4 fields
    # above PLUS the 2 new ones below – avoids retyping the same 4 fields twice.
    id: int
    course_id: int
    class Config:
        from_attributes = True

```

Every remaining schema file (workshop.py, event.py, session.py, point.py) follows this exact Create/Out pattern — one pair of classes per table, matching the fields listed in Section 6. To save space, they aren't all repeated here, but the pattern never changes: list only what the client should send in the Create schema, and never include password_hash or similar secrets in any Out schema.

8. dependencies.py — Auth, Explained Line by Line

8.1 What "dependency injection" means, in plain words

In FastAPI, a "dependency" is just a normal Python function that FastAPI runs automatically BEFORE your route's own code, and whose return value it hands to you as an argument. Writing `Depends(get_current_user)` as a parameter means: "before running my function, run `get_current_user()` for me, and give me whatever it returns." This is how the same 5 lines of "check who's logged in" logic gets reused across 30+ routes without copy-pasting it into every single one.

8.2 The full file

```
import os, datetime
# os -> reads environment variables (JWT_SECRET etc.)
# datetime -> used to calculate token expiry times

from fastapi import Depends, HTTPException, status
# Depends -> the dependency-injection marker explained above
# HTTPException -> how you deliberately stop a request and return an error response
# status -> a collection of readable names for HTTP status codes (e.g. status.HTTP_401_UNAUTHORIZED
# instead of just typing the number 401 – easier to read and less error-prone)

from fastapi.security import OAuth2PasswordBearer
# A helper that knows how to read the "Authorization: Bearer <token>" header out of a request
# and also makes the /docs page show a proper "Authorize" login button automatically.

from sqlalchemy.orm import Session
# The TYPE of the `db` object – used here just for type hints, so your editor can autocomplete.

from jose import jwt, JWTError
# jwt -> functions to create (encode) and read (decode) JWT tokens
# JWTError -> the specific error type raised if a token is invalid/expired/tampered with

from passlib.context import CryptContext
# The tool that actually performs bcrypt password hashing and verification.

from app.database import get_db
from app.models.user import User

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
# Sets up passlib to specifically use the bcrypt algorithm. "deprecated=auto" means if you
# ever add a newer algorithm later, passlib can auto-upgrade old hashes transparently.

oauth2_scheme = OAuth2PasswordBearer(tokenUrl="/api/auth/login")
# Registers where the /docs page should send login requests to get a token – this is what
# makes the little "Authorize" lock icon appear on the FastAPI docs page.

JWT_SECRET = os.getenv("JWT_SECRET")
JWT_ALGORITHM = os.getenv("JWT_ALGORITHM", "HS256")
# The "HS256" here is a FALLBACK default, only used if JWT_ALGORITHM isn't set in .env at all.

def hash_password(password: str) -> str:
    # Takes a plain-text password (e.g. "SecurePass1") and returns its bcrypt hash – a long,
    # irreversible string that's safe to store in the database.
    return pwd_context.hash(password)

def verify_password(plain: str, hashed: str) -> bool:
    # Checks whether a plain-text password, when hashed, matches a previously-stored hash.
    # Used during login: we NEVER un-hash the stored password, we just compare hashes.
    return pwd_context.verify(plain, hashed)

def create_access_token(data: dict, expires_minutes: int = 1440):
    # Builds a signed JWT token. `data` is typically {"user_id": ..., "role": ...}.
    to_encode = data.copy()
    # Copy the dict so we don't accidentally modify the caller's original dictionary.
    expire = datetime.datetime.utcnow() + datetime.timedelta(minutes=expires_minutes)
    # Calculates the exact moment this token should stop being valid.
    to_encode.update({"exp": expire})
    # "exp" is a JWT-standard field name – jose automatically checks it on every decode and
    # rejects the token once this time has passed, so we don't need to check it ourselves.
```

```

return jwt.encode(to_encode, JWT_SECRET, algorithm=JWT_ALGORITHM)
# Signs and returns the final token string, e.g. "eyJhbGciOiIi..".

def get_current_user(token: str = Depends(oauth2_scheme), db: Session = Depends(get_db)) -> User:
    # This is itself a dependency that uses TWO other dependencies:
    # Depends(oauth2_scheme) -> automatically extracts the token string from the
    # Authorization header of the incoming request
    # Depends(get_db) -> gives us a database session to look the user up with
    try:
        payload = jwt.decode(token, JWT_SECRET, algorithms=[JWT_ALGORITHM])
        # Verifies the token's signature (proving it was really issued by our server and
        # hasn't been tampered with) and also checks it hasn't expired. If anything is
        # wrong, this line raises a JWTError, caught below.
        user_id = payload.get("user_id")
        # Pulls the user_id we originally embedded when the token was created.
    except JWTError:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="Invalid token")
        # Immediately stops processing and sends back a 401 Unauthorized response.

    user = db.query(User).filter(User.id == user_id).first()
    # Looks up the actual, current User row in the database by that id – we re-fetch fresh
    # data every request rather than trusting the (possibly outdated) info inside the token.
    if not user:
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED, detail="User not found")
        # Covers the edge case where a valid token exists for a user that was later deleted.
    return user
    # Whatever this function returns becomes the value FastAPI hands to any route that
    # wrote `current_user: User = Depends(get_current_user)`.

def require_role(*allowed_roles):
    # `*allowed_roles` means this function accepts any number of role-name arguments, e.g.
    # require_role("president", "vice_president"). It then returns ANOTHER function
    # (`checker`) that FastAPI will actually call as the real dependency – this pattern is
    # called a "dependency factory", and it's how we get a reusable, configurable check.
    def checker(current_user: User = Depends(get_current_user)):
        # Notice this itself depends on get_current_user – so calling require_role(...)
        # automatically ALSO enforces "must be logged in" for free.
        if current_user.role not in allowed_roles:
            raise HTTPException(status_code=status.HTTP_403_FORBIDDEN, detail="Not allowed")
        return current_user
    return checker

```

8.3 Bonus: the Committee Access System check (Section 7 of the baseline)

For the extra rule where President/VP/Secretary can grant one committee edit access to another committee's section, add this reusable helper to the same file:

```

from app.models.point import AccessPermission

def has_edit_access(db: Session, current_user: User, section_name: str) -> bool:
    # Presidents/VPs/Secretaries and the section's own default-owning committee always
    # pass – everyone else needs an explicit AccessPermission row with can_edit=True.
    if current_user.role in ("president", "vice_president", "secretary"):
        return True
    permission = db.query(AccessPermission).filter_by(
        section_name=section_name, committee_id=current_user.committee_id).first()
    return permission is not None and permission.can_edit
    # Returns True/False – routes call this manually (not via Depends) when they need the
    # EXTRA layer of checking on top of the normal role check, e.g. inside courses.py's
    # create_course function, right after the require_role(...) dependency has already run.

```

9. Core Backend Logic — The 5 Trickiest Rules, Explained and Coded

This is where the AICHE baseline's written rules turn into actual working code. Each of these 5 functions belongs in `app/core_logic.py` and gets imported by whichever router needs it.

9.1 Sequential course unlock (baseline Rule 1)

The rule, in the baseline's own words: "A part cannot be opened until the previous part is completed." In database terms: before letting a student open `course_part #3`, we must check whether their `course_progress` row for `course_part #2` has status = "completed".

```
from sqlalchemy.orm import Session
from app.models.course import CoursePart, CourseProgress

def is_part_unlocked(db: Session, user_id: int, part: CoursePart) -> bool:
    if part.order_index == 1:
        # The very first part of any course is always open – there's nothing before it
        # to have completed, so we skip the database lookup entirely.
        return True

    previous = db.query(CoursePart).filter(
        CoursePart.course_id == part.course_id,
        CoursePart.order_index == part.order_index - 1
    ).first()
    # Finds the ONE part that comes immediately before this one in the same course, by
    # looking for order_index - 1 (e.g. if this part is #3, we look for #2).

    progress = db.query(CourseProgress).filter_by(
        user_id=user_id, course_part_id=previous.id
    ).first()
    # Looks for this specific student's progress row on that previous part.

    return progress is not None and progress.status == "completed"
    # Returns True only if a progress row exists AND its status is exactly "completed".
    # If the student never even started the previous part, `progress` is None, and this
    # safely returns False without crashing.
```

Why this function doesn't care whether completion happened online or via an admin-confirmed offline attendance sheet: both paths write the exact same status = "completed" value onto the same `course_progress` row — the baseline's own rule, "Completion = Online Completion OR Admin-confirmed Offline Completion", becomes true automatically just by both code paths writing to the same field.

9.2 Total Points — always calculated, never stored as an editable number (Rule 6)

The baseline is explicit: "Total Points is NOT entered manually — it is calculated automatically." The reason this matters: if Total Points were its own column that someone could directly edit, it could silently drift out of sync with the actual history of tasks/attendance/events. Instead, we treat `point_entries` as the single source of truth, and Total Points is just a live calculation over it.

```
from sqlalchemy import func
from app.models.point import PointEntry

def get_total_points(db: Session, user_id: int) -> int:
    total = db.query(func.sum(PointEntry.points)).filter(
        PointEntry.user_id == user_id
    ).scalar()
    # func.sum(...) tells the DATABASE to add up all matching `points` values itself
    # (fast, even with thousands of rows) rather than pulling every row into Python and
    # summing there. .scalar() extracts the single resulting number from the query result.
    return total or 0
    # If the user has zero point_entries at all, SQL SUM() returns None, not 0 – the
    # `or 0` here converts that None into a normal 0 so callers don't need a special case.

def add_point_entry(db: Session, user_id: int, source_type: str, source_id: int,
    points: int, reason: str):
    # The ONE function every other part of the app calls whenever points should change –
    # centralizing this means the point_entries table is always written to consistently.
    entry = PointEntry(user_id=user_id, source_type=source_type, source_id=source_id,
```

```
        points=points, reason=reason)
db.add(entry)
# Note: this function deliberately does NOT call db.commit() itself – the calling
# route (e.g. the course-part-completion route) is expected to commit once, after
# all of its own changes are ready, so everything saves together or not at all.
```

9.3 Certificate generation (Sections 18 & 21)

Two different trigger conditions, matching the baseline's two certificate types:

- **Course certificate** — issued once (a) every required course_part shows status = "completed", AND (b) the saved grade on the final_exam part meets or exceeds whatever pass threshold you decide (the baseline leaves the exact number up to you).
- **Workshop certificate** — issued once workshop_attendance shows "present" for every required session; this is an Attendance Certificate, not grade-based, matching Rule 2 of the baseline exactly.

```
def check_and_issue_course_certificate(db: Session, user_id: int, course_id: int):
    parts = db.query(CoursePart).filter_by(course_id=course_id).all()
    # Every part that belongs to this course.

    all_completed = all(
        db.query(CourseProgress).filter_by(
            user_id=user_id, course_part_id=part.id, status="completed"
        ).first() is not None
        for part in parts
    )
    # `all(... for part in parts)` is Python's way of checking "is this true for EVERY
    # item in the list?" – if even one part is missing a completed progress row, this is False.

    if not all_completed:
        return None
    # Stop here – nothing to issue yet.

    # (PDF generation code using reportlab/fpdf2, and uploading to Cloudinary, goes here –
    # omitted for brevity; it's plain file-handling code, not business logic.)
    certificate = Certificate(user_id=user_id, type="course", source_id=course_id,
                             pdf_url="<generated pdf url>")

    db.add(certificate)
    db.commit()
    return certificate
```

9.4 Session visibility (Rule 3)

The baseline's rule: "A Member sees only their own committee's Sessions plus any Public Sessions — except QC, who sees every committee's Sessions."

```
from app.models.session import CommitteeSession

def get_visible_sessions(db: Session, current_user):
    if current_user.role == "committee_admin" and current_user.committee.name == "QC":
        return db.query(CommitteeSession).all()
    # The one deliberate exception carved out in Section 33 of the baseline: QC skips
    # the filter entirely and sees literally everything.

    return db.query(CommitteeSession).filter(
        (CommitteeSession.visibility == "public") |
        (CommitteeSession.committee_id == current_user.committee_id)
    ).all()
    # The "|" here means OR at the database level: return any row where EITHER the
    # visibility is public, OR the committee_id matches the current user's own committee.
    # A Member from Marketing, for example, will never see a committee_specific session
    # created by Technical, because neither condition is true for them.
```

9.5 Feedback report → delete flow (Rules 7 & 8)

Two-step process by design: OC can flag a comment, but only President/VP can actually approve its removal — and removing it also removes the offending account, atomically.

```
def report_feedback(db: Session, feedback_id: int, reporter):
    fb = db.query(Feedback).get(feedback_id)
    fb.is_reported = True
    fb.reported_by = reporter.id
    db.commit()
    # Nothing is hidden or deleted yet – this just adds it to the President/VP's review
    # queue (any route listing feedback for review simply filters WHERE is_reported=True).
```

```
def delete_reported_feedback(db: Session, feedback_id: int, approver):
    fb = db.query(Feedback).get(feedback_id)
    author = db.query(User).get(fb.user_id)

    fb.is_deleted = True
    fb.deleted_by = approver.id
    db.delete(author)
    # Both changes are prepared here but NOT yet saved to the database.

    db.commit()
    # This single commit() call saves BOTH changes together. If anything went wrong between
    # the two operations above (e.g. the process crashed), NEITHER change would be saved –
    # this "all or nothing" behavior is what the baseline's Rule 8 ("delete the Comment +
    # the Account together") actually requires, and SQLAlchemy gives it to us for free as
    # long as we don't call commit() twice.
```

10. Routers — Full Code, Every Line Explained

A "router" in FastAPI is a mini version of the whole app, focused on one feature — you build it separately, then "include" it into the main app in main.py (Section 11). Every other router file not fully repeated here (workshops.py, events.py, database_routes.py, reports.py, feedback.py, access.py) follows the exact same pattern shown in these two.

10.1 routers/auth.py

```
from fastapi import APIRouter, Depends, HTTPException
# APIRouter -> the class used to define a group of related routes before plugging them
# into the main app.

from sqlalchemy.orm import Session
from app.database import get_db
from app.models.user import User
from app.schemas.user import UserCreate, UserLogin, UserOut, Token
from app.dependencies import hash_password, verify_password, create_access_token, get_current_user

router = APIRouter()
# Creates the router object. Every @router.get/@router.post below registers one endpoint
# onto this object — nothing is actually live yet until main.py includes it.

@router.post("/signup", response_model=UserOut, status_code=201)
# @router.post(...) -> this function handles POST requests to this path
# "/signup" -> combined with the prefix set in main.py ("/api/auth"), the
# full path becomes POST /api/auth/signup
# response_model=UserOut -> FastAPI will filter/shape whatever we return to match
# exactly the UserOut schema (no password_hash leaks out)
# status_code=201 -> "201 Created" is the correct HTTP status for successfully
# creating a new resource (the default would be 200)
def signup(user: UserCreate, db: Session = Depends(get_db)):
    # `user: UserCreate` -> FastAPI reads the request's JSON body, validates it against the
    # UserCreate schema, and hands you a ready-to-use Python object.
    # `db: Session = Depends(get_db)` -> a fresh database session for this one request.

    if db.query(User).filter(User.email == user.email).first():
        raise HTTPException(400, "Email already registered")
        # Stops early with a clear error if this email is already taken, instead of letting
        # the database throw a confusing low-level "unique constraint violated" error.

    new_user = User(
        full_name=user.full_name,
        username=user.username,
        email=user.email,
        password_hash=hash_password(user.password),
        # Notice: we hash the password HERE, right before storing it — the plain-text
        # password from the request never touches the database.
    )
    db.add(new_user)
    # Stages the new row to be inserted (not saved yet).
    db.commit()
    # Actually sends the INSERT to PostgreSQL and saves it permanently.
    db.refresh(new_user)
    # Reloads new_user from the database, filling in fields the database generated itself
    # (like the auto-incremented `id` and the default `created_at` timestamp).
    return new_user
    # FastAPI converts this SQLAlchemy object into JSON using the UserOut schema.

@router.post("/login", response_model=Token)
def login(credentials: UserLogin, db: Session = Depends(get_db)):
    user = db.query(User).filter(User.email == credentials.email).first()

    if not user or not verify_password(credentials.password, user.password_hash):
        raise HTTPException(401, "Incorrect email or password")
        # Deliberately using the SAME error message whether the email doesn't exist or the
        # password is wrong — this avoids revealing to an attacker whether a given email
        # is even registered.

    token = create_access_token({"user_id": user.id, "role": user.role})
    return {"access_token": token, "token_type": "bearer"}
```

```
# Returning a plain dict here is fine – FastAPI will still validate/shape it against  
# the Token response_model declared above.
```

```
@router.get("/me", response_model=UserOut)  
def get_me(current_user: User = Depends(get_current_user)):  
    # The entire body is just `return current_user` – all the actual work (reading the  
    # token, verifying it, looking up the user) already happened inside get_current_user().  
    return current_user
```

10.2 routers/courses.py

```
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session
from app.database import get_db
from app.models.course import Course, CoursePart, CourseProgress
from app.models.user import User
from app.schemas.course import CourseCreate, CourseOut, CoursePartCreate, CoursePartOut
from app.dependencies import get_current_user, require_role
from app.core_logic import is_part_unlocked, add_point_entry

router = APIRouter()

@router.get("/", response_model=list[CourseOut])
def list_courses(db: Session = Depends(get_db)):
    # No Depends(get_current_user) here at all – this route is intentionally public,
    # matching the baseline's rule that Guests can view course listings.
    return db.query(Course).all()

@router.post("/", response_model=CourseOut, status_code=201)
def create_course(
    course: CourseCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_role(
        "committee_admin", "president", "vice_president", "secretary")),
    # This single line enforces the entire role rule: only these 4 roles can even reach
    # the code below – everyone else gets an automatic 403 before this function body runs.
):
    new_course = Course(**course.dict(), created_by=current_user.id)
    # course.dict() turns the validated CourseCreate object into a plain Python dict, e.g.
    # {"title": "...", "description": "...", ...}. The ** unpacks that dict into keyword
    # arguments, equivalent to writing Course(title=..., description=..., ...) by hand.
    # created_by=current_user.id is added separately since it never comes from the client.
    db.add(new_course)
    db.commit()
    db.refresh(new_course)
    return new_course

@router.post("/{course_id}/parts", response_model=CoursePartOut, status_code=201)
def add_part(
    course_id: int,
    # FastAPI automatically pulls this straight out of the URL, e.g. /courses/5/parts -> 5.
    part: CoursePartCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_role("committee_admin", "president")),
):
    new_part = CoursePart(**part.dict(), course_id=course_id)
    db.add(new_part)
    db.commit()
    db.refresh(new_part)
    return new_part

@router.post("/course-parts/{part_id}/complete")
def complete_part(
    part_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(get_current_user),
    # Just needs to be logged in – any Student or Member can call this for themselves.
):
    part = db.query(CoursePart).get(part_id)
    # .get(part_id) is a SQLAlchemy shortcut specifically for looking something up by
    # primary key – slightly faster/clearer than .filter(CoursePart.id == part_id).first()

    if not is_part_unlocked(db, current_user.id, part):
        raise HTTPException(403, "Previous part not completed yet")
    # This is exactly where Rule 1 (Section 9.1) gets enforced – the route itself
    # doesn't know the unlock LOGIC, it just asks the helper function and obeys it.

    progress = CourseProgress(
        user_id=current_user.id, course_part_id=part_id,
        status="completed", completion_type="online",
    )
```

```
db.add(progress)

if part.type == "task":
    add_point_entry(db, current_user.id, "task", progress.id, 2, "Task submitted on time")
    # Only Task-type parts earn points on completion, per the baseline's scoring rules
    # (Sections 35-39) – Lectures and Quizzes don't have a points value listed there.

db.commit()
# ONE commit at the end saves both the new progress row AND the new point entry (if
# any) together – this matters: if we crash between adding progress and adding the
# point entry, neither would be half-saved.
return {"message": "Part completed"}
```

11. main.py — Every Line Explained

```
from fastapi import FastAPI
# The main class – creating an instance of this IS your entire web application.

from fastapi.middleware.cors import CORSMiddleware
# CORS = Cross-Origin Resource Sharing. Browsers block a website on one domain from
# calling an API on a different domain unless the API explicitly allows it – this
# middleware is what grants that permission.

from app.routers import (auth, courses, workshops, events, sessions,
                        points, database_routes, reports, feedback, access)
# Imports every router MODULE (not just specific functions) – we need the whole module
# because we're about to reference `auth.router`, `courses.router`, etc. below.

app = FastAPI(title="AICHE Backend API")
# Creates the actual application object. `title` shows up at the top of the /docs page.

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    # "*" means "allow requests from any website" – fine for early development/testing,
    # but before going live you should replace this with your real frontend's exact URL
    # (e.g. ["https://aiche-cu.com"]) so random other websites can't call your API from
    # a user's browser using their logged-in session.
    allow_methods=["*"],
    # Allows all HTTP methods (GET, POST, PUT, DELETE, etc.)
    allow_headers=["*"],
    # Allows all request headers, including our custom Authorization header.
)

app.include_router(auth.router, prefix="/api/auth", tags=["Auth"])
# `prefix` is prepended to every path inside that router – so auth.py's "/signup" becomes
# the real path "/api/auth/signup". `tags` is purely cosmetic: it groups these endpoints
# under a labeled section on the /docs page, making it easier to browse.
app.include_router(courses.router, prefix="/api/courses", tags=["Courses"])
app.include_router(workshops.router, prefix="/api/workshops", tags=["Workshops"])
app.include_router(events.router, prefix="/api/events", tags=["Events"])
app.include_router(sessions.router, prefix="/api/sessions", tags=["Sessions"])
app.include_router(points.router, prefix="/api/points", tags=["Points"])
app.include_router(database_routes.router, prefix="/api/committees", tags=["Database"])
app.include_router(reports.router, prefix="/api/reports", tags=["Reports"])
app.include_router(feedback.router, prefix="/api/feedback", tags=["Feedback"])
app.include_router(access.router, prefix="/api/access-permissions", tags=["Access"])

@app.get("/")
def root():
    # A simple, un-prefixed route at the very base URL – mainly useful so that when you
    # (or Render's health-check system) visit the bare API URL, you get a clear confirmation
    # the server is actually running, instead of a confusing 404.
    return {"status": "AICHE backend is running"}
```

Run it locally with `uvicorn app.main:app --reload`: `app.main` tells Uvicorn to look inside the `app` package's `main.py` file, and `:app` tells it exactly which variable inside that file is the FastAPI application object. `--reload` makes it automatically restart whenever you save a code change — very useful during development, but remove it in production.

12. Alembic Migrations & Seed Data — Explained in Depth

12.1 What a "migration" actually is

A migration is a small file describing ONE change to your database's structure — "add a courses table", "add a grade column to certificates", etc. — written as two Python functions: `upgrade()` (apply the change) and `downgrade()` (undo it). Alembic keeps every migration file chained together in order (each one records which one came right before it), so it always knows exactly what a database needs to catch up to your latest models, whether it's your laptop, a teammate's laptop, or the live production database.

12.2 First-time setup (once per project)

```
alembic init alembic
# Creates the alembic/ folder and alembic.ini file shown in Section 4. This is a
# one-time setup command, never run again after the first time.
```

Then open a `alembic/env.py` and make two edits: import your Base object, and point `target_metadata` at it, so Alembic knows which models to compare the database against:

```
from app.database import Base
from app.models import * # ensures every model file has actually been imported

target_metadata = Base.metadata
# Before this line, target_metadata = None (the default), which would make Alembic
# unable to auto-detect any of your table definitions.
```

12.3 Every time you add or change a model

```
alembic revision --autogenerate -m "add courses table"
# --autogenerate -> Alembic compares your current models/ code against the live database's
# actual structure, and writes a new file in alembic/versions/ containing
# the SQL commands needed to make the database match your code.
# -m "add courses table" -> a short human-readable message describing this change, saved
# into the generated filename and file contents.

alembic upgrade head
# Actually RUNS that migration file (and any earlier ones not yet applied) against your
# real database. "head" means "the most recent migration in the chain".
```

Always open and skim the auto-generated file before running upgrade — `autogenerate` is usually right, but it can occasionally miss things (like renaming a column, which it may see as "delete one column, add a different one" instead).

12.4 `seed.py` — creating starter data

Migrations create empty tables — they don't fill them with data. For this project, you need two pieces of starter data before the system is usable: the fixed list of committees, and the very first President account (since only a President can normally grant other roles, someone has to exist before anyone else can be promoted).

```
from app.database import SessionLocal
from app.models.user import Committee, User
from app.dependencies import hash_password

db = SessionLocal()
# Opens a database session directly — this script runs OUTSIDE of any FastAPI request, so
# there's no Depends(get_db) available; we call SessionLocal() ourselves instead.

committees = ["Marketing", "OC", "Technical", "Web Development", "PR", "QC", "General Committee"]
for name in committees:
    if not db.query(Committee).filter_by(name=name).first():
        # Checks whether this committee already exists before inserting — this makes the
        # script "idempotent", meaning you can safely run it more than once without
        # creating duplicate rows.
        db.add(Committee(name=name, login_email=f"aiche.{name.lower()}@gmail.com"))
db.commit()

if not db.query(User).filter_by(role="president").first():
    db.add(User(
```

```
        full_name="Head President", username="president", email="president@aiche.com",
        password_hash=hash_password("ChangeMe123!"), role="president",
    ))
db.commit()

print("Seed complete.")
```

Run once with `python seed.py` right after your first alembic upgrade head. Log in with that President account immediately afterward and change the password.

13. Full API Reference — Every Endpoint, Every Field Explained

Test every one of these through /docs, Postman, or Thunder Client — no frontend needed. [Role] shows the minimum required role from Depends(require_role(...)).

Auth

POST /api/auth/signup — Create an account. full_name/username/email/password are all required; role always defaults to student and cannot be set by the client. [Public]

REQUEST BODY:

```
{
  "full_name": "Ahmed Ali",
  "username": "ahmed_ali",
  "email": "ahmed@example.com",
  "password": "SecurePass1"
}
```

RESPONSE:

```
201 Created
{
  "id": 5, "full_name": "Ahmed Ali", "username": "ahmed_ali",
  "email": "ahmed@example.com", "role": "student",
  "committee_id": null, "created_at": "2026-09-10T10:00:00"
}
```

POST /api/auth/login — Exchange email+password for a JWT access token, valid for ACCESS_TOKEN_EXPIRE_MINUTES. [Public]

REQUEST BODY:

```
{
  "email": "ahmed@example.com",
  "password": "SecurePass1"
}
```

RESPONSE:

```
200 OK
{
  "access_token": "eyJhbGciOi...", "token_type": "bearer"
}
```

GET /api/auth/me — Returns the profile of whichever user the Bearer token belongs to. [Any logged-in user]

RESPONSE:

```
200 OK
{
  "id": 5, "full_name": "Ahmed Ali", "role": "student", ...
}
```

Courses

GET /api/courses — List every course (no auth needed). [Public]

RESPONSE:

```
200 OK
[
  {"id": 1, "title": "Intro to ChemE", "level": "Beginner", ...}
]
```

POST /api/courses — Create a course; created_by is taken from the logged-in admin automatically. [Committee Admin+]

REQUEST BODY:

```
{
  "title": "Intro to Chemical Engineering",
  "description": "...",
  "image_url": "https://...",
  "instructor": "Dr. Ahmed",
  "level": "Beginner",
  "duration": "4 weeks"
}
```

```
RESPONSE:
201 Created
{
  "id": 1, "title": "Intro to Chemical Engineering", "created_by": 3, ...
}
```

POST /api/courses/{id}/parts — Add one Lecture/Task/Quiz/Final Exam. order_index determines unlock order. [Committee Admin+]

```
REQUEST BODY:
{
  "type": "lecture",
  "order_index": 1,
  "title": "Lecture 1: Basics",
  "content_url": "https://video-link"
}
```

```
RESPONSE:
201 Created
{
  "id": 10, "course_id": 1, "type": "lecture", "order_index": 1, ...
}
```

POST /api/courses/{id}/enroll — Register the logged-in user for this course (no content is unlocked by enrollment alone — course_parts still follow the unlock rule). [Student/Member]

```
RESPONSE:
200 OK
{
  "message": "Enrolled successfully"
}
```

POST /api/courses/course-parts/{id}/complete — Marks a part completed (online path) — rejected with 403 if the previous part isn't done yet. [Student/Member]

```
RESPONSE:
200 OK
{
  "message": "Part completed"
}

403 Forbidden
{
  "detail": "Previous part not completed yet"
}
```

POST /api/courses/course-parts/{id}/attendance — Offline completion path — an admin marks a specific student Present, which sets their course_progress the same way the online path does. [Admin]

```
REQUEST BODY:
{
  "user_id": 5,
  "status": "present"
}
```

```
RESPONSE:
200 OK
{
  "message": "Attendance recorded, part marked completed"
}
```

GET /api/courses/{id}/certificate — Returns the certificate PDF link, if it has been generated (see Section 9.3 for the trigger conditions). [Student/Member]

```
RESPONSE:
200 OK
{
  "pdf_url": "https://cloudinary.../certificate_5_course_1.pdf"
}

404 Not Found (not yet earned)
{
  "detail": "Certificate not yet issued"
}
```

Workshops

GET /api/workshops — List workshops. [Public]

```
RESPONSE:
200 OK
[
  {"id": 1, "title": "Intro to Excel", "instructor": "Sara", ...}
]
```

POST /api/workshops — Create a workshop. [Admin]

```
REQUEST BODY:
{
  "title": "Intro to Excel",
  "description": "...",
  "image_url": "https://...",
  "instructor": "Sara"
}

RESPONSE:
201 Created
{
  "id": 1, "title": "Intro to Excel", "created_by": 3, ...
}
```

POST /api/workshops/{id}/sessions — Add a session to a workshop. [Admin]

```
REQUEST BODY:
{
  "title": "Session 1",
  "description": "...",
  "date": "2026-10-01T18:00:00",
  "mode": "online",
  "location_or_link": "https://zoom.us/..."
}

RESPONSE:
201 Created
{
  "id": 4, "workshop_id": 1, "title": "Session 1", ...
}
```

POST /api/workshops/sessions/{id}/attendance — Mark Present/Absent for one student on one session. [Admin]

```
REQUEST BODY:
{
  "user_id": 5,
  "status": "present"
}

RESPONSE:
200 OK
{
  "message": "Attendance recorded"
}
```

GET /api/workshops/{id}/certificate — Attendance-based certificate — no grade field, unlike course certificates. [Student/Member]

```
RESPONSE:
200 OK
{
  "pdf_url": "https://cloudinary.../certificate_5_workshop_1.pdf"
}
```

Events & Site Visits

GET /api/events — List events/site visits; status (Upcoming/Closed/Finished) is calculated on the fly from `date` and `registration\_deadline`, not stored. [Public]

```
RESPONSE:
200 OK
[
```

```
{ "id": 1, "type": "event", "title": "Career Fair", "status": "upcoming", ... }
]
```

POST /api/events — Create an event or site visit. [Admin]

REQUEST BODY:

```
{
  "type": "event",
  "title": "Career Fair",
  "description": "...",
  "date": "2026-11-01T09:00:00",
  "location": "Main Hall",
  "registration_deadline": "2026-10-28T23:59:00"
}
```

RESPONSE:

201 Created

```
{
  "id": 1, "type": "event", "title": "Career Fair", ...
}
```

POST /api/events/{id}/register — Register for an event; rejected once registration_deadline has passed. [Student/Member]

RESPONSE:

200 OK

```
{
  "message": "Registered successfully"
}
```

400 Bad Request

```
{
  "detail": "Registration is closed"
}
```

POST /api/events/{id}/attendance — Mark Attended/Absent — automatically adds a point_entries row per Section 37's scoring table. [Admin]

REQUEST BODY:

```
{
  "user_id": 5,
  "status": "attended"
}
```

RESPONSE:

200 OK

```
{
  "message": "Attendance recorded", "points_earned": 2
}
```

Sessions

GET /api/sessions — Returns only sessions visible to the caller — see Section 9.4 for the exact visibility rule. [Member+]

```
RESPONSE:
200 OK
[
  {"id": 2, "title": "Weekly Sync", "committee_id": 3, "visibility": "committee_specific"}
]
```

POST /api/sessions — Create a committee session. [Committee Admin]

```
REQUEST BODY:
{
  "title": "Weekly Sync",
  "description": "...",
  "date": "2026-09-15T17:00:00",
  "visibility": "committee_specific"
}

RESPONSE:
201 Created
{
  "id": 2, "committee_id": 3, "title": "Weekly Sync", ...
}
```

GET /api/sessions/{id} — Session details plus the full scoring table for every member with a session_scoring row. [Member+]

```
RESPONSE:
200 OK
{
  "id": 2, "title": "Weekly Sync",
  "scoring": [
    {"user_id": 5, "attendance_status": "on_time", "task_status": "on_time", "bonus_points": 1}
  ]
}
```

POST /api/sessions/{id}/scoring — Save one member's attendance/task/engagement/bonus for this session; each field maps to a specific point value from Sections 35-39 and writes a point_entries row per field. [Admin/QC]

```
REQUEST BODY:
{
  "user_id": 5,
  "attendance_status": "on_time",
  "task_status": "on_time",
  "engagement_status": "engaged",
  "bonus_points": 1
}

RESPONSE:
200 OK
{
  "message": "Scoring saved", "points_added": 5
}
```

Points

GET /api/points/{user_id}/total — Live SUM over point_entries — see Section 9.2. [Member+]

```
RESPONSE:
200 OK
{
  "user_id": 5, "total_points": 27
}
```

GET /api/points/{user_id}/history — Every individual point_entries row, newest first — this is exactly the "Point History" list from Section 41. [Member+]

```
RESPONSE:
200 OK
[
```

```
  {"points": 2, "reason": "Task submitted on time", "created_at": "2026-09-08T..."},
  {"points": -1, "reason": "Absent without excuse", "created_at": "2026-09-01T..."}
]
```

Committee Database

GET /api/committees/{id}/database — Any logged-in user can **VIEW** any committee's rows (Rule 4 — view-all). [All view]

```
RESPONSE:
200 OK
[
  {"id": 1, "row_data": {"place": "Cairo Print Shop", "phone": "0100...", "price": "500 EGP"}}
]
```

POST /api/committees/{id}/database — Only the **OWNING** committee (or a committee explicitly granted access via /api/access-permissions) can add a row. [Owning committee only]

```
REQUEST BODY:
{
  "row_data": {"place": "Cairo Print Shop", "phone": "01001234567", "price": "500 EGP"}
}
```

```
RESPONSE:
201 Created
{
  "id": 1, "committee_id": 3, "row_data": {...}
}
```

```
403 Forbidden
{
  "detail": "Not allowed"
}
```

Monthly Reports

POST /api/reports/members/{id}/monthly-report — QC uploads a report directly into a specific Member's profile. [QC]

```
REQUEST BODY:
{
  "file_url": "https://cloudinary.../report_sept.pdf",
  "month": "2026-09"
}

RESPONSE:
201 Created
{
  "id": 3, "member_id": 5, "month": "2026-09", "uploaded_by": 8
}
```

Feedback

POST /api/feedback — Leave feedback on a course/workshop/event/site visit the user participated in.

[Student/Member]

```
REQUEST BODY:
{
  "target_type": "course",
  "target_id": 1,
  "comment": "Really helpful course!"
}

RESPONSE:
201 Created
{
  "id": 12, "user_id": 5, "target_type": "course", "target_id": 1, "comment": "..."
}
```

POST /api/feedback/{id}/report — Flags a comment for President/VP review — does not delete it (Section 9.5). [OC]

```
RESPONSE:
200 OK
{
  "message": "Feedback reported", "is_reported": true
}
```

DELETE /api/feedback/{id} — Approves removal — deletes the comment AND the author's account together, in one transaction. [President/VP]

```
RESPONSE:
200 OK
{
  "message": "Comment and account deleted"
}
```

Access Control

POST /api/access-permissions — Grants or revokes one committee's edit access to one named section.

[President/VP/Secretary]

```
REQUEST BODY:
{
  "section_name": "courses",
  "committee_id": 1,
  "can_edit": true
}

RESPONSE:
200 OK
{
  "section_name": "courses", "committee_id": 1, "can_edit": true, "granted_by": 2
}
```

14. Running & Deploying for Free

Run locally

```
uvicorn app.main:app --reload
# Starts the server on http://localhost:8000 and watches your files for changes.

# Then open http://localhost:8000/docs in your browser — this is an interactive page,
# auto-generated from your route definitions and Pydantic schemas, where you can click
# "Try it out" on any endpoint, fill in the fields, and see the real request/response.
```

Deploy free on Render, step by step

- Push your entire backend/ folder to a GitHub repository (make sure .env is NOT included — check your .gitignore).
- Create a free account at render.com, then click New → Web Service, and connect your GitHub repo.
- Build Command: pip install -r requirements.txt
- Start Command: uvicorn app.main:app --host 0.0.0.0 --port \$PORT (--host 0.0.0.0 means "accept connections from anywhere", and \$PORT is a variable Render sets automatically — never hard-code a port number here).
- In Render's dashboard, add DATABASE_URL, JWT_SECRET, JWT_ALGORITHM, and ACCESS_TOKEN_EXPIRE_MINUTES as Environment Variables — this is Render's own secure equivalent of your local .env file.
- Once deployed, open Render's built-in Shell tab for your service and run alembic upgrade head once, then python seed.py once, against the live database.
- Your API is now live at a URL like https://aiche-backend.onrender.com — test https://aiche-backend.onrender.com/docs to confirm.

15. Backend Split — 2 Members

Same feature-vertical split as before, mapped onto the exact Python files from this guide — each member owns their own model files, schema files, and router file end-to-end, which is what keeps merge conflicts rare.

Shared foundation (both members, Days 1–3)

- All of models/ (Section 6) + the first Alembic revision, built and run together
- dependencies.py (Section 8) + routers/auth.py (Section 10.1)
- core_logic.py's shared helper add_point_entry() — agree its exact function signature here even if Member B writes the full implementation, since Member A's code calls it too

Member A — Learning Track	Member B — Community & Operations Track
models/course.py, models/workshop.py	models/event.py, models/session.py, models/point.py
schemas/course.py, schemas/workshop.py	schemas/event.py, schemas/session.py, schemas/point.py
routers/courses.py, routers/workshops.py	routers/events.py, routers/sessions.py, routers/points.py, routers/database_routes.py, routers/reports.py, routers/feedback.py, routers/access.py
Core logic owned: is_part_unlocked(), check_and_issue_course_certificate()	Core logic owned: get_total_points(), add_point_entry(), get_visible_sessions(), report_feedback(), delete_reported_feedback()

16. Backend Split — 3 Members

Member A — Learning	Member B — Committee Ops	Member C — Points/Admin
models/course.py, models/workshop.py	models/event.py, models/session.py (+ CommitteeDatabaseRow in point.py)	PointEntry, MonthlyReport, Feedback, AccessPermission (rest of point.py)
routers/courses.py, routers/workshops.py	routers/events.py, routers/sessions.py, routers/database_routes.py	routers/points.py, routers/reports.py, routers/feedback.py, routers/access.py

Member C should start wiring real add_point_entry() calls against live data produced by A and B by **Week 2** — not the last week — since Points, Feedback moderation, and Access Control all read across the whole system.

17. Git Workflow for This Stack

Alembic migrations form a linear chain — each generated file records which one came immediately before it (its "down_revision"). This is the one place Python's migration tool needs a specific team rule that Prisma-style tools don't.

Step	Action
1	main = deployable, develop = shared integration branch, feature/<module> = one branch per module.
2	Rebase onto develop daily: git fetch origin && git rebase origin/develop
3	Only run alembic revision --autogenerate right before opening your PR, on an up-to-date local database — if two people generate migrations from outdated local databases at the same time, their chains conflict when merged.
4	If two migration files DO end up conflicting after a merge: run alembic merge heads -m "merge migrations" to create one combined file that chains both branches together, then alembic upgrade head.

Step	Action
5	PR reviewed by at least one other member before merging, squash-merged into develop.
6	Everyone else runs alembic upgrade head locally right after pulling develop.
7	Weekly joint integration test (Section 19) before merging develop → main.

18. Common Beginner Mistakes & How to Avoid Them

Mistake	Why it's a problem	Fix
Forgetting <code>db.commit()</code>	Your data disappears the moment the request ends — <code>db.add()</code> only STAGES a change, it doesn't save it.	Always pair every <code>db.add()/db.delete()</code> with a <code>db.commit()</code> before returning.
Returning the raw SQLAlchemy model instead of using <code>response_model</code>	Can accidentally leak internal fields like <code>password_hash</code> to the client.	Always declare <code>response_model=...</code> on every route and trust it to filter fields.
Putting business logic (like the unlock check) directly inside a router function	Makes it hard to reuse and hard to test; also easy to accidentally duplicate slightly-different logic in two different routers.	Keep it in <code>core_logic.py</code> and import it — see Section 9.
Hard-coding secrets (DB password, JWT secret) directly in code	Anyone who sees your GitHub repo sees your secrets too.	Always read from <code>.env</code> via <code>os.getenv()</code> , and never commit <code>.env</code> to Git.
Trusting fields like <code>`role`</code> or <code>`created_by`</code> from the request body	A malicious client could sign up and set role: "president" themselves.	Only ever set these from server-side values like <code>current_user.id</code> — see the Create schemas in Section 7, which never include these fields at all.
Running alembic revision <code>--autogenerate</code> on a stale local database	Produces a migration file with conflicting assumptions about what already exists.	Always alembic upgrade head first to sync your local database, THEN generate.
One giant <code>models.py</code> / <code>routers.py</code> file	Two teammates editing the same huge file cause constant Git merge conflicts.	Keep the one-file-per-module structure from Section 4, even if it feels like "too many small files" at first.

19. Final Integration Checklist

Run all of this through /docs, Postman, or Thunder Client — no UI required.

- POST `/api/auth/signup` then `/api/auth/login` → confirm a valid JWT comes back.
- Enroll a test user in a Course → confirm `course_parts` unlock only in order (Section 9.1).
- Submit a Task → confirm a `point_entries` row is created and `/api/points/{id}/total` reflects it (Section 9.2).
- Complete all parts + pass the Final Exam → confirm `/api/courses/{id}/certificate` returns a generated PDF URL (Section 9.3).
- GET `/api/sessions` as a Member → confirm only their committee's + Public sessions come back, and that a QC admin sees everything (Section 9.4).
- Register for an Event, mark Attended → confirm points update.
- Call each endpoint as every role (missing token, Student, Member, Committee Admin, President/VP/Secretary) → confirm 401/403 happen exactly where they should — cross-check against Section 2's per-role table.
- POST `.../report` as OC → DELETE the feedback as President → confirm the comment and the account are both gone in one transaction (Section 9.5).
- POST `/api/access-permissions` granting one committee edit access → confirm only that committee can now write to the section (Section 8.3).
- Run through the MVP feature list (Section 60 of the original baseline) endpoint-by-endpoint and confirm nothing was missed.

Source: based on the approved "AICHE Website — Description Website Final" baseline. All tools referenced (Python, FastAPI, SQLAlchemy, Alembic, PostgreSQL, Pydantic, passlib, python-jose, Uvicorn, Render free tier, Neon/Supabase free tier, GitHub) are free and/or fully open source at the time of writing. This document intentionally excludes all frontend/UI content.